PostgreSQL Logical Replication

Table of Contents

Learning Objectives

By the end of this module, you will be able to:

- Create publications and subscriptions for logical replication
- Apply row and column filters to replicate data subsets
- Plan and execute zero-downtime PostgreSQL major version upgrades
- Understand logical replication limitations and workarounds
- Monitor logical replication health and resolve conflicts
- Design multi-subscriber architectures

How Logical Replication Works

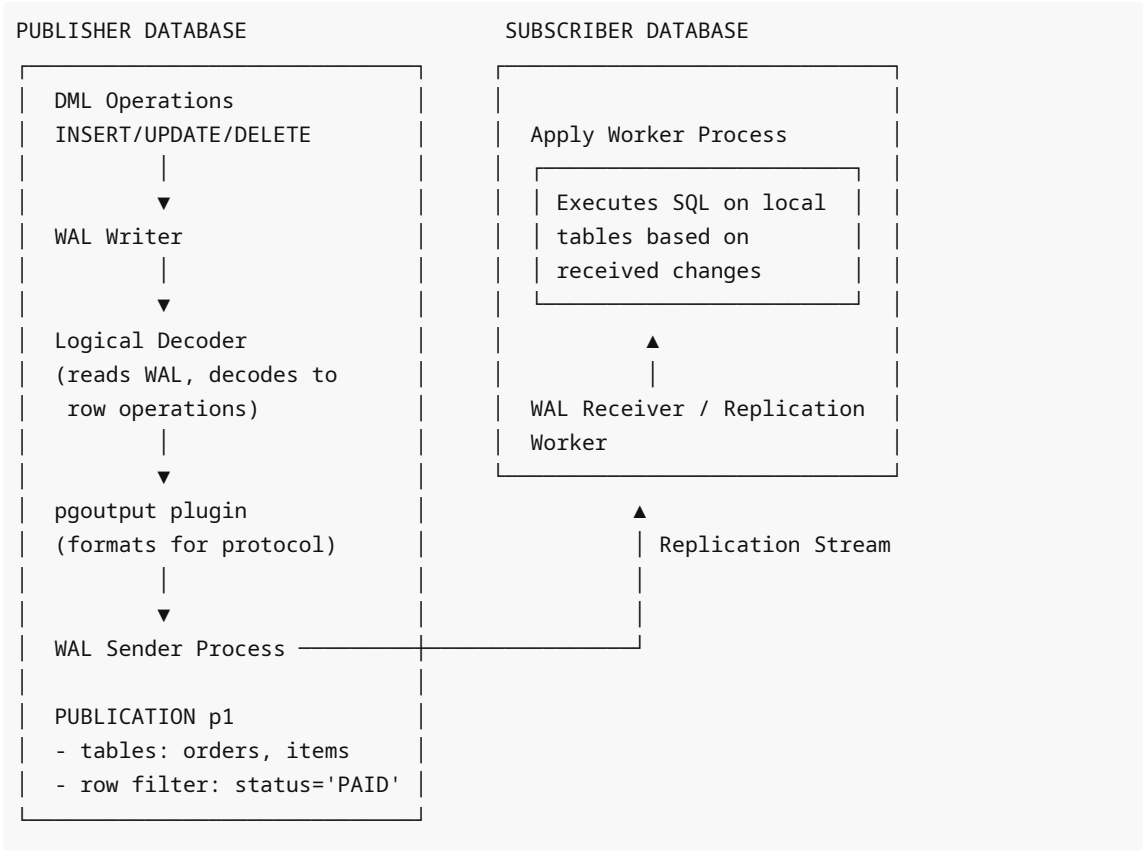
Logical replication uses **logical decoding** to extract row-level changes from WAL and send them to subscribers. Instead of binary block copies, it sends decoded INSERT/UPDATE/DELETE operations.

Logical Decoding Pipeline

```
WAL Files ──> Logical Decoding ──> Output Plugin ──> Subscriber

Step 1: WAL records generated by DML
Step 2: Logical decoder reads WAL
Step 3: pgoutput plugin formats as protocol messages
Step 4: Walsender streams to subscriber
Step 5: Apply worker executes DML on subscriber
```

Component Architecture



Publications

A publication is a set of changes from a table or group of tables that are made available for replication.

Creating Publications

```
-- Publish all changes for all tables
CREATE PUBLICATION pub_all FOR ALL TABLES;

-- Publish specific tables
CREATE PUBLICATION pub_orders FOR TABLE orders, order_items, customers;

-- Publish specific operations only
CREATE PUBLICATION pub_inserts_only
  FOR TABLE events
  WITH (publish = 'insert');    -- Only replicate INSERTs, not UPDATE/DELETE

-- Publish insert and update but not delete
CREATE PUBLICATION pub_no_delete
  FOR TABLE products
  WITH (publish = 'insert, update');

-- Publish with row filter (PostgreSQL 15+)
CREATE PUBLICATION pub_paid_orders
  FOR TABLE orders
```

```
  WHERE (status = 'PAID');

-- Publish specific columns (PostgreSQL 16+)
CREATE PUBLICATION pub_public_data
  FOR TABLE users (id, username, email, created_at);
  -- Excludes: password_hash, ssn, credit_card
```

Managing Publications

```
-- View publications
SELECT pubname, puballtables, pubinsert, pubupdate, pubdelete, pubtruncate
FROM pg_publication;

-- View tables in a publication
SELECT schemaname, tablename
FROM pg_publication_tables
WHERE pubname = 'pub_orders';

-- Add a table to existing publication
ALTER PUBLICATION pub_orders ADD TABLE shipments;

-- Remove a table
ALTER PUBLICATION pub_orders DROP TABLE old_archive;

-- Change publication options
ALTER PUBLICATION pub_orders SET (publish = 'insert, update, delete');

-- Drop publication
DROP PUBLICATION pub_orders;

-- Publication for schemas (PostgreSQL 15+)
CREATE PUBLICATION pub_schema FOR TABLES IN SCHEMA public, analytics;
```

Subscriptions

A subscription is the downstream side — it connects to a publication and receives and applies changes.

Creating Subscriptions

```
-- Basic subscription
CREATE SUBSCRIPTION sub_orders
  CONNECTION 'host=primary-db.example.com port=5432 dbname=production
user=replicator password=secret'
  PUBLICATION pub_orders;

-- Subscription with custom slot name
CREATE SUBSCRIPTION sub_orders
  CONNECTION 'host=192.168.1.100 port=5432 dbname=mydb user=replicator'
  PUBLICATION pub_orders
```

```
  WITH (slot_name = 'analytics_slot');

-- Subscription without copying initial data (for append-only tables)
CREATE SUBSCRIPTION sub_events
  CONNECTION 'host=192.168.1.100 port=5432 dbname=mydb user=replicator'
  PUBLICATION pub_events
  WITH (copy_data = false);

-- Subscription without creating a replication slot (manual slot management)
CREATE SUBSCRIPTION sub_manual
  CONNECTION 'host=192.168.1.100 port=5432 dbname=mydb user=replicator'
  PUBLICATION pub_orders
  WITH (create_slot = false, slot_name = 'my_existing_slot');
```

Managing Subscriptions

```
-- View subscriptions
SELECT subname, subenabled, subslotname, subpublications
FROM pg_subscription;

-- Disable a subscription (pause replication)
ALTER SUBSCRIPTION sub_orders DISABLE;

-- Re-enable
ALTER SUBSCRIPTION sub_orders ENABLE;

-- Refresh publication tables (after adding tables to publication)
ALTER SUBSCRIPTION sub_orders REFRESH PUBLICATION;

-- Refresh with no initial data copy for new tables
ALTER SUBSCRIPTION sub_orders REFRESH PUBLICATION WITH (copy_data = false);

-- Change connection string
ALTER SUBSCRIPTION sub_orders
  CONNECTION 'host=new-primary.example.com port=5432 dbname=mydb user=replicator';

-- Drop subscription (also drops the replication slot on publisher by default)
DROP SUBSCRIPTION sub_orders;

-- Drop without dropping the slot (if publisher is unreachable)
DROP SUBSCRIPTION sub_orders WITH (slot_name = NONE);
```

Required Publisher-Side Configuration

```
# postgresql.conf on publisher
wal_level = logical             # Required for logical replication (not replica)
max_replication_slots = 10
max_wal_senders = 10
```

```
-- Grant required permissions to replication user
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'secret';

-- For PostgreSQL 14 and earlier:
GRANT SELECT ON ALL TABLES IN SCHEMA public TO replicator;

-- For PostgreSQL 15+:
GRANT pg_read_all_data TO replicator;

-- Grant access to specific tables only:
GRANT SELECT ON orders, customers, products TO replicator;
```

```
# pg_hba.conf on publisher
host    all             replicator      192.168.1.0/24          scram-sha-256
```

Row Filtering

Row filtering allows a publication to replicate only rows matching a condition (PostgreSQL 15+).

```
-- Replicate only paid orders
CREATE PUBLICATION pub_paid
  FOR TABLE orders
  WHERE (status = 'PAID');

-- Replicate only recent data
CREATE PUBLICATION pub_recent
  FOR TABLE events
  WHERE (created_at > now() - INTERVAL '30 days');

-- Multi-table with different filters
CREATE PUBLICATION pub_regional
  FOR TABLE orders WHERE (region = 'EU'),
      customers WHERE (country IN ('DE', 'FR', 'UK'));

-- Replicate only non-deleted records (soft delete pattern)
CREATE PUBLICATION pub_active
  FOR TABLE products WHERE (deleted_at IS NULL);

-- Filter based on user tier
CREATE PUBLICATION pub_premium
  FOR TABLE user_activity WHERE (user_tier = 'premium');
```

Row Filter Limitations

```
-- Filters CANNOT use:
-- - Subqueries
-- - User-defined functions (non-immutable)
-- - System columns
```

```
-- - Volatile functions like now() in a static sense

-- ALLOWED in row filters:
-- - Comparison operators
-- - Boolean operators
-- - Immutable functions
-- - Constants and literal values

-- Workaround for time-based filters: use a computed column
ALTER TABLE events ADD COLUMN is_recent BOOLEAN
  GENERATED ALWAYS AS (created_at > '2024-01-01') STORED;

CREATE PUBLICATION pub_recent_events FOR TABLE events WHERE (is_recent = true);
```

Column Filtering

Column filtering publishes only specified columns (PostgreSQL 16+).

```
-- Replicate only safe columns (exclude PII/sensitive data)
CREATE PUBLICATION pub_analytics
  FOR TABLE users (id, username, created_at, country, plan_type);
  -- Excludes: email, password_hash, ssn, phone, address

-- Combine with row filter
CREATE PUBLICATION pub_public_orders
  FOR TABLE orders (id, status, total_amount, created_at)
  WHERE (status IN ('COMPLETED', 'SHIPPED'));

-- Product catalog without cost data
CREATE PUBLICATION pub_product_catalog
  FOR TABLE products (id, name, description, price, category_id, stock_qty);
  -- Excludes: cost_price, supplier_id, margin

-- View column publication info
SELECT pubname, schemaname, tablename, attnames
FROM pg_publication_tables
WHERE pubname = 'pub_analytics';
```

Column Filtering Requirements

```
-- IMPORTANT: The replica identity must be included in published columns
-- If not using DEFAULT replica identity, the key columns must be published

-- Check replica identity
SELECT relname, relreplident FROM pg_class WHERE relname = 'users';
-- d = default (primary key), n = nothing, f = full, i = index

-- If using custom replica identity, that column must be in publication:
ALTER TABLE users REPLICA IDENTITY USING INDEX idx_users_uuid;
```

```
CREATE PUBLICATION pub_users FOR TABLE users (uuid, username, email);
-- uuid must be included because it's the replica identity
```

Zero-Downtime Major Version Migrations

Logical replication is the standard approach for upgrading PostgreSQL major versions (e.g., 13 → 16) with minimal downtime.

Migration Architecture

```
PHASE 1: Parallel Operation
┌──────────────────┐     Logical Replication     ┌──────────────────┐
│  PG 13 (old)     │ ──────────────────────────> │  PG 16 (new)     │
│  PRIMARY         │                             │  (subscriber)    │
│  wal_level=      │                             │  Read-Write      │
│  logical         │                             │  (but idle)      │
└──────────────────┘                             └──────────────────┘
        │                                                │
    App writes                                    Catches up and
    here                                          stays in sync

PHASE 2: Cutover (1-5 minutes downtime)
1. Drain connections to old primary
2. Wait for subscriber to fully catch up (lag = 0)
3. Stop app, verify no more writes to old
4. Disable subscription on new
5. Update app connection string to PG 16
6. Resume app on PG 16
```

Step-by-Step Migration Procedure

```
# Step 1: Set up PG 16 instance (new server or same server different port)
# Install PostgreSQL 16, initialize data directory
sudo apt-get install postgresql-16
sudo -u postgres /usr/lib/postgresql/16/bin/initdb -D /var/lib/postgresql/16/main

# Step 2: Create schema on PG 16 (logical replication does NOT replicate DDL)
# Export schema from PG 13
pg_dump --schema-only -d mydb -h old-server > schema.sql

# Apply to PG 16
psql -d mydb -h new-server -f schema.sql
```

```
-- Step 3: On PG 13 (old primary): configure for logical replication
-- Requires restart if not already set
ALTER SYSTEM SET wal_level = 'logical';
-- Restart PG 13

-- Create publication for all tables
```

```
CREATE PUBLICATION migration_pub FOR ALL TABLES;

-- Create replication user
CREATE ROLE replicator WITH REPLICATION LOGIN PASSWORD 'migpass';
GRANT pg_read_all_data TO replicator;
```

```
# pg_hba.conf on PG 13: allow connection from PG 16 server
host    all          replicator    <new-server-ip>/32    scram-sha-256
```

```
-- Step 4: On PG 16 (new subscriber): create subscription
CREATE SUBSCRIPTION migration_sub
  CONNECTION 'host=old-server port=5432 dbname=mydb user=replicator
password=migpass'
  PUBLICATION migration_pub;

-- Monitor initial sync progress
SELECT relid::regclass, state, processed, estimated_rows
FROM pg_stat_subscription_stats;

-- Wait for all tables to complete initial copy
SELECT * FROM pg_subscription_rel WHERE srsubstate != 'r';
-- 'r' = ready (synced), 'i' = init, 'd' = data copy, 'f' = finished copy, 's' =
synchronized
```

```
-- Step 5: Monitor ongoing replication lag
-- On PG 16:
SELECT received_lsn, latest_end_lsn,
       now() - latest_end_time AS lag_time
FROM pg_stat_subscription;

-- On PG 13: check what PG 16 has consumed
SELECT slot_name, confirmed_flush_lsn,
       pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS bytes_behind
FROM pg_replication_slots
WHERE slot_name = 'migration_sub';
```

```
# Step 6: CUTOVER PROCEDURE (maintenance window)

# a) Notify users of brief maintenance
# b) Gracefully stop application writes to PG 13
# c) Verify replication lag is zero:
psql -h old-server -c "SELECT pg_wal_lsn_diff(pg_current_wal_lsn(),
confirmed_flush_lsn) FROM pg_replication_slots WHERE slot_name = 'migration_sub';"
# Should be 0 or very close

# d) On PG 16: disable subscription (makes it read-write)
psql -h new-server -c "ALTER SUBSCRIPTION migration_sub DISABLE;"
# Then drop it:
psql -h new-server -c "DROP SUBSCRIPTION migration_sub;"
```

```
# e) Migrate sequences (NOT replicated by logical replication!)
# Export sequence values from PG 13:
psql -h old-server -d mydb -t -c "
SELECT 'SELECT setval(' || quote_literal(sequence_schema || '.' || sequence_name) ||
', ' || last_value || ');'
FROM information_schema.sequences s
JOIN pg_sequences ps ON ps.sequencename = s.sequence_name
ORDER BY sequence_schema, sequence_name;
" > set_sequences.sql

# Apply on PG 16:
psql -h new-server -d mydb -f set_sequences.sql

# f) Update application connection string to new server
# g) Test application

# h) Cleanup: drop publication on PG 13 (after verification)
psql -h old-server -c "DROP PUBLICATION migration_pub;"
```

Monitoring Logical Replication

```
-- Subscription status
SELECT
    subname,
    subenabled,
    subslotname,
    subpublications
FROM pg_subscription;

-- Worker status per subscription
SELECT
    subname,
    pid,
    relid::regclass AS table_name,
    received_lsn,
    last_msg_send_time,
    last_msg_receipt_time,
    latest_end_lsn,
    latest_end_time
FROM pg_stat_subscription;

-- Per-table sync state
SELECT
    s.subname,
    c.relname AS table_name,
    CASE srsubstate
        WHEN 'i' THEN 'Initialize'
        WHEN 'd' THEN 'Data copy'
        WHEN 'f' THEN 'Finished copy'
```

```
        WHEN 's' THEN 'Synchronized'
        WHEN 'r' THEN 'Ready'
    END AS state
FROM pg_subscription_rel sr
JOIN pg_subscription s ON s.oid = sr.srsubid
JOIN pg_class c ON c.oid = sr.srrelid;

-- Replication slot on publisher
SELECT slot_name, active, confirmed_flush_lsn,
       pg_wal_lsn_diff(pg_current_wal_lsn(), confirmed_flush_lsn) AS bytes_behind
FROM pg_replication_slots
WHERE slot_type = 'logical';
```

Conflict Handling

Logical replication can encounter conflicts (e.g., INSERT of a row that already exists on subscriber).

```
-- Check for conflicts in logs
-- Look for ERROR messages like:
-- ERROR: duplicate key value violates unique constraint "orders_pkey"
-- CONTEXT: logical replication target relation "public.orders"

-- Resolve conflict by skipping the offending LSN
-- Find the LSN from the error message or pg_stat_subscription

-- Skip a transaction on subscriber (PostgreSQL 15+)
ALTER SUBSCRIPTION sub_orders SKIP (lsn = '0/4A1234BC');

-- For older PostgreSQL, disable and manually resolve:
ALTER SUBSCRIPTION sub_orders DISABLE;
-- Fix the conflicting data manually
-- Re-enable
ALTER SUBSCRIPTION sub_orders ENABLE;
```

Common Mistakes

1. **Setting `wal_level = replica` instead of `logical`** — logical replication won't work
2. **Forgetting to create the schema on subscriber** — tables must exist before subscription
3. **Not handling sequences** — sequences are NOT replicated; must be manually migrated
4. **Not handling DDL changes** — ALTER TABLE on publisher breaks replication
5. **Using `TRUNCATE` without `publish = 'truncate'`** — TRUNCATE not included by default in older versions
6. **Dropping subscription without dropping slot** — orphaned slot on publisher accumulates WAL
7. **Large transactions causing apply delays** — one huge transaction blocks subsequent smaller ones
8. **Not granting SELECT to replication user** — subscription worker fails with permission denied
9. **Replication identity not set** — UPDATE/DELETE on tables without primary key fail
10. **Circular replication** — bidirectional replication without conflict resolution causes infinite loops

---

Best Practices

1. Always set `REPLICA IDENTITY` on tables without primary keys before replication
2. Monitor replication slot lag to prevent WAL disk exhaustion
3. For migrations: test the cutover procedure in staging before production
4. Apply DDL changes carefully: add columns to subscriber first (before publisher)
5. Use connection pooling for subscriber connections
6. Set `wal_level = logical` only when needed — it has slightly higher overhead than `replica`
7. Drop publications and subscriptions when no longer needed
8. Use row filtering to reduce replication volume and protect sensitive data

---

Performance Considerations

```
# Publisher: tuning for logical decoding
max_worker_processes = 16          # Enough for apply workers
logical_decoding_work_mem = 64MB    # Memory for decoding large transactions
```

```
-- Subscriber: check apply worker performance
SELECT * FROM pg_stat_activity WHERE backend_type = 'logical replication apply
worker';

-- Large transactions: parallel apply (PostgreSQL 15+)
-- Configure on subscription:
ALTER SUBSCRIPTION sub_orders SET (streaming = on);  -- Stream large transactions
ALTER SUBSCRIPTION sub_orders SET (parallel_apply_worker_per_subscription = 4);
```

Interview Questions

Q1: What is the difference between a publication and a subscription?

A: A publication is defined on the publisher (source) database and specifies which tables and operations to expose for replication. A subscription is defined on the subscriber (destination) and connects to a specific publication. One publication can have many subscribers; one subscriber can subscribe to many publications on different servers.

Q2: Why doesn't logical replication replicate DDL?

A: Logical replication decodes row-level changes from WAL, but DDL changes produce different WAL records that the pgoutput plugin doesn't translate into portable DDL statements. Schema synchronization would be complex (handling cross-version differences, conditional DDL, etc.). PostgreSQL deliberately keeps DDL out of scope, requiring users to manage schema changes manually or use third-party tools.

Q3: What is `REPLICA IDENTITY` and why does it matter for logical replication?

A: Replica identity determines what information is included in WAL records for UPDATE and DELETE operations so the subscriber can identify which row to update/delete. Options: `DEFAULT` (primary key columns), `FULL` (all column values), `NOTHING` (no identity — UPDATE/DELETE not replicable), `INDEX` (specific unique index). Tables without primary keys default to `NOTHING` and must be explicitly configured before updates/deletes can be replicated.

Q4: How do you handle a zero-downtime PostgreSQL major version upgrade?

A: Use logical replication: (1) Set up new PG instance. (2) Export schema to new instance. (3) Configure `wal_level=logical` on old. (4) Create publication on old. (5) Create subscription on new (initial data copy). (6) Monitor until replication lag is near zero. (7) Cutover: stop writes to old, verify lag=0, disable/drop subscription, update connection strings. (8) Migrate sequences manually.

Q5: What happens to sequences during logical replication?

A: Sequences are NOT replicated by logical replication. Only DML on regular tables is replicated. After migration cutover, you must manually set sequence values to at least the maximum values they had on the source, plus a safety buffer. Failure to do this causes duplicate key errors when the application inserts new rows using the old sequence values.

Q6: How do you handle conflicts in logical replication?

A: Conflicts (e.g., a subscriber already having the row being inserted) are logged as errors and can stop the apply worker. Resolution approaches: (1) Skip the conflicting transaction using `ALTER SUBSCRIPTION sub SKIP (lsn = 'X/Y')` in PG15+. (2) Manually fix the conflicting data on subscriber and re-enable subscription. (3) Use `synchronous_commit = off` to reduce blocking. Prevention: ensure subscriber data is clean before subscribing.

Q7: Can you do bidirectional logical replication?

A: Yes, but it's dangerous without conflict resolution. Both servers publish and subscribe to each other. Conflicts arise when the same row is modified on both sides. Use with extreme caution and only with application-level conflict prevention (e.g., disjoint rows, last-write-wins with timestamps). PostgreSQL 16 adds built-in bidirectional support improvements.

Q8: What is `streaming = on` for subscriptions?

A: By default, logical replication buffers large transactions to disk and applies them atomically when the transaction commits. With `streaming = on`, the subscriber starts applying the transaction incrementally as changes arrive, reducing latency and disk usage for large transactions. Available in PostgreSQL 14+.

Exercises and Solutions

Exercise 1: Set Up Logical Replication for Analytics

```
-- On publisher: create publication for analytics team
-- They need: orders (no amounts), products (no cost), customers (no PII)

-- PG 16+ with column filtering:
CREATE PUBLICATION pub_analytics
  FOR TABLE orders (id, status, created_at, customer_id, item_count),
      products (id, name, category, description),
      customers (id, account_type, country, created_at);

-- On subscriber (analytics database):
CREATE SUBSCRIPTION sub_analytics
  CONNECTION 'host=prod-db port=5432 dbname=production user=analytics_repl'
  PUBLICATION pub_analytics;
```

Exercise 2: Implement Row Filter for Regional Data

```
-- Publisher in US, subscriber in EU
-- EU subscriber should only get EU customers and their orders

CREATE PUBLICATION pub_eu
  FOR TABLE customers WHERE (region = 'EU'),
      orders WHERE (billing_country IN ('DE','FR','UK','IT','ES','NL','PL'));
```

Exercise 3: Migration Cutover Checklist

Create a runbook for migration cutover:

```
PRE-CUTOVER CHECKLIST:
[ ] New server running PG target version
[ ] Schema applied to new server
[ ] Subscription created, initial copy complete
[ ] Replication lag < 1 second sustained
[ ] Sequences migrated script prepared
[ ] Application connection change prepared
[ ] Rollback plan documented

CUTOVER STEPS:
[ ] T-5min: Notify users of maintenance window
[ ] T-0: Set application to maintenance mode / stop writes
[ ] T+30s: Verify replication lag = 0 bytes
[ ] T+1min: Disable subscription on new server
[ ] T+2min: Run set_sequences.sql on new server
[ ] T+3min: Update application connection string
[ ] T+4min: Run smoke tests on new server
[ ] T+5min: Remove maintenance mode
[ ] T+30min: Monitor error rates, verify no issues
[ ] T+24h: Drop publication on old server
```

Cross-References

- 01_replication_overview.md — Physical vs logical comparison
- 02_streaming_replication.md — Physical replication setup
- 14_Security/03_row_level_security.md — RLS with logical replication
- 15_Backup_Recovery/04_pitr.md — WAL-based recovery